

GSencrypt: Implementação de um Sistema de Gestão de Credenciais Baseado em Arquitetura de Conhecimento Zero e Conformidade com a LGPD

Anny Gabriely Souza do Nascimento, Antonio Luiz Lins Neto, Fábio Yuuki Saruwataru

Orientador: Prof. Dr. Fabiano Bezerra Menegídio
Universidade de Mogi das Cruzes (UMC)

Resumo

O gerenciamento centralizado de identidades e credenciais no ciberespaço apresenta desafios intrínsecos de segurança, frequentemente atuando como um Ponto Único de Falha (Single Point of Failure). Em um cenário de constantes violações de dados, projetar sistemas que retenham credenciais sem expô-las aos provedores de nuvem tornou-se um imperativo de engenharia. Este artigo descreve a arquitetura, o modelo criptográfico e a adequação legal do projeto GSencrypt, um cofre de senhas institucional pautado pelos princípios de *Privacy by Design* e pela tríade da Segurança da Informação (Confidencialidade, Integridade e Disponibilidade). Através da implementação de uma Arquitetura de Conhecimento Zero (Zero-Knowledge), o ecossistema garante que apenas o titular dos dados consiga derivar as chaves simétricas necessárias para a decifragem do cofre, blindando os registros contra exfiltrações promovidas por agentes internos, como administradores de banco de dados. Adicionalmente, o sistema incorpora fluxos nativos de adequação ao Artigo 18 da Lei Geral de Proteção de Dados (LGPD), assegurando transparência estruturada em JSON, portabilidade automatizada e fluxos de exclusão em cascata. O projeto utiliza o padrão Monolito Modularizado suportado pelo *framework* Django 6.0.4, orquestrado via Docker, apresentando-se como um protótipo robusto para ambientes corporativos sujeitos a auditorias de conformidade estritas.

Palavras-Chave: Criptografia, Zero-Knowledge, LGPD, Segurança da Informação, Django.

1 Introdução e Problema de Pesquisa

A convergência dos serviços digitais tornou o armazenamento de credenciais um alvo primário para cibercriminosos. Cofres de senhas tradicionais, quando não implementam criptografia isolada na camada da aplicação, centralizam um volume massivo de dados sensíveis em formato legível para o banco de dados. Essa centralização atua como um Ponto Único de Falha (*Single Point of Failure* - SPoF), no qual a comprometimento de uma única credencial administrativa do Sistema Gerenciador de Banco de Dados (SGBD) resulta no vazamento em massa de informações críticas.

Nesse contexto, o problema de pesquisa que norteia este trabalho pode ser sintetizado na seguinte questão: *Como mitigar a exposição de dados sensíveis ao próprio provedor de nuvem ou infraestrutura de aplicação e, concorrentemente, garantir a conformidade técnica e automatizada com o Artigo 18 da Lei Geral de Proteção de Dados (LGPD)?* Para solucionar tal desafio, o projeto GSencrypt foi concebido a partir de uma abordagem de defesa em profundidade, garantindo que o dado chegue anonimizado ou cifrado à camada de persistência.

2 Arquitetura de Software e Stack Tecnológica

O ecossistema do GSencrypt foi desenhado sob o padrão arquitetural de Monolito Modularizado. Esta escolha metodológica prioriza a rastreabilidade e a segregação lógica de responsabilidades, evitando as complexidades prematuras de uma arquitetura baseada em microsserviços, enquanto mantém fronteiras de contexto estritamente delimitadas.

A *stack* tecnológica é sustentada por um ambiente minimalista e isolado em contêineres Docker, executando o sistema operacional *slim* combinado com Python 3.12 e o *framework* Django na versão 6.0.4. A segregação do código-fonte ocorre através de submódulos (apps) autônomos, categorizados da seguinte forma:

- **accounts:** Responsável pela gestão de ciclo de vida de usuários, higienização de identificadores (*case-insensitive*) e provisionamento do perfil criptográfico.
- **authentication:** Submódulo dedicado aos mecanismos de *login*, geração de tokens, verificação de múltiplos fatores e controle absoluto do tempo de sessão e inatividade.
- **vault:** O coração criptográfico da aplicação. Gerencia o CRUD do cofre de credenciais e as operações matemáticas de cifragem simétrica de mão dupla.
- **audit:** Mecanismo de subrotina acoplado para registro de *logs* transacionais no padrão *Write Once, Read Many* (WORM).
- **recovery:** Gerenciamento de contingências e *reset* de credenciais via comunicação *out-of-band* (SMTP).
- **lgpd:** Orquestrador de privacidade, gerindo o consentimento dinâmico e respondendo aos direitos legais dos titulares.

3 Mecanismos de Segurança e Camada Criptográfica (Zero-Knowledge)

A espinha dorsal de segurança do GSencrypt fundamenta-se em diretrizes internacionais estabelecidas pela *Open Worldwide Application Security Project* (OWASP) e especificações matemáticas rigorosas.

3.1 Armazenamento de Senhas e Hashing

A proteção contra ataques *offline*, como o uso de processamento paralelo massivo via GPUs ou ASICs, é mitigada mediante a substituição do algoritmo padrão do Django pela classe `CustomArgon2Hasher`, parametrizada via `settings.py`. Baseando-se na RFC 9106 para Argon2id, os fatores de complexidade operam com os seguintes delimitadores: custo de tempo (`time_cost = 2`), custo de memória (`memory_cost = 65536` KB, correspondente a 64MB) e paralelismo de *threads* (`parallelism = 1`). Esta parametrização atrasa o *throughput* de verificação para o adversário, inviabilizando abordagens de força bruta ou ataques de dicionário.

3.2 Proteção de Dados em Repouso

Para prover a funcionalidade *Zero-Knowledge*, todos os segredos transitados no módulo `vault` são cifrados em tempo de execução via *Advanced Encryption Standard* (AES) com chaves de 256 bits, utilizando a implementação Fernet da biblioteca `cryptography`. A chave

mestra de encriptação não reside no banco de dados, sendo derivada dinamicamente por intermédio da variável de ambiente `SECRET_KEY`. Como resultado prático, administradores do PostgreSQL (*root*) possuem acesso apenas a sequências alfanuméricas cifradas (textos cifrados não determinísticos), preservando a integridade arquitetural em cenários de extração pura.

3.3 Autenticação em Duas Etapas (MFA) e Prevenção de Abuso

A superfície de autenticação exige, obrigatoriamente, um fluxo de dois estágios totalmente desacoplados no módulo `authentication`. Inicialmente ocorre o *Pre-auth Check* da senha mestra. Em caso de sucesso parcial, o sistema exige o desafio de Múltiplos Fatores de Autenticação (MFA) via algoritmo TOTP (*Time-based One-Time Password*) suportado pela biblioteca PyOTP.

Uma via de contingência via SMTP (utilizando o provedor Brevo) está ativa para cenários de perda do token. Para evitar o esgotamento de recursos via disparos excessivos (*spam/throttling*), aplicou-se um algoritmo de *cooldown* de 30 segundos entre solicitações. Complementarmente, a biblioteca `django-axes` atua no bloqueio imediato de endereços de rede (IPs) após sucessivas falhas de submissão do desafio multifator.

3.4 Trilha de Auditoria Imutável (WORM)

A integridade dos rastros sistêmicos (*SIA - Security, Integrity, Availability*) foi desenhada no modelo `AuditLog`. Para evitar manipulação de registros de log a partir da própria interface administrativa do Django ou via *scripts* locais, o *Object-Relational Mapping* (ORM) foi encapsulado. Os métodos padrão `save()` e `delete()` foram matematicamente sobrepostos: caso ocorra a tentativa de mutação de uma tupla que já possui uma chave primária registrada (ou a requisição de deleção), o código lança uma exceção `PermissionDenied`. Tal arquitetura força o comportamento *Write-Only* no banco de dados, configurando os *logs* como provas imutáveis adequadas a perícia forense.

4 Engenharia de Conformidade Legal (LGPD)

O desenvolvimento seguro sob as diretrizes de *Privacy by Design* não trata a privacidade como um complemento, mas como o requisito funcional central. O módulo `lgpd` foi arquitetado para satisfazer integralmente o Artigo 18 da Lei nº 13.709/2018 (LGPD), transformando obrigações jurídicas abstratas em execuções de *software* concretas:

- **Direito de Acesso e Transparência:** O subsistema disponibiliza a rota `consultar_meus_dados`, fornecendo um *payload* JSON com toda a árvore relacional de atividades e de configurações armazenadas, integralmente higienizadas para consumo direto.
- **Direito à Portabilidade:** Na função `exportar_meus_dados`, o *software* constrói um arquivo empacotado que agrega o histórico de consentimento e as chaves cifradas. Este arquivo é forçado como um *HTTP attachment* diretamente no navegador, evitando gravações intermediárias no servidor local e cumprindo a portabilidade irrestrita.
- **Direito ao Esquecimento:** Para a função `excluir_minha_conta`, aciona-se a deleção em cascata (*CASCADE*) por meio do ORM do Django. O processo extirpa o identificador do usuário primário, e, conseqüentemente, toda e qualquer credencial ou sub-registro a ele associado é irreversivelmente removida da base de dados física e lógica, satisfazendo a remoção permanente perante o marco regulatório.

- **Gestão Ativa de Consentimento:** Utilizando um *gatekeeper* (barreira) na base lógica, o usuário tem a prerrogativa de revogar sua anuência. Se o estado de aceite for alterado para nulo ou falso na tabela respectiva, requisições HTTP posteriores originadas deste portador sofrerão interrupção, paralisando todo o processamento de visualização do cofre (*vault*) instantaneamente, sendo restauradas somente após nova chancela explícita.

5 Análise de Vulnerabilidades e Remediação de Débito Técnico

A validação de uma estrutura DevSecOps engloba o reconhecimento e remediação de falhas herdadas (*débitos técnicos*), especialmente em códigos concebidos por métodos de Vibecoding (auxílio não supervisionado de LLMs). Baseando-se no relatório técnico de auditoria, quatro eixos de contramedidas arquiteturais foram postos em produção para chancelar a viabilidade do produto.

1. **Erradicação de Servidores Síncronos em Produção:** O comando nativo `runserver`, inadequado para o tráfego externo, foi sumariamente substituído pelo servidor WSGI de nível industrial, *Gunicorn*. A configuração adotada conta com a diretiva `--limit-request-line` e múltiplos *workers* assíncronos, mitigando *crashes* por *buffer overflow* na leitura do cabeçalho.
2. **Prevenção de Negação de Serviço (DoS):** Um *Middleware* customizado, denominado `PayloadSizeLimitMiddleware`, atua como primeira camada defensiva na *pipeline* HTTP. Esse interceptador protege os recursos limitados da máquina avaliando antecipadamente o tamanho do cabeçalho e rejeitando arquivos gigantes ou *payloads* inflados (*large blobs*) que possam exaurir a memória RAM ou CPU do ambiente *host* por alocação assimétrica de recursos.
3. **Criptografia de Trânsito Obrigatória (TLS):** Com o auxílio e isolamento da rede via *bridges* do Docker Compose e suporte externo ao Nginx, a diretiva `SECURE_SSL_REDIRECT = True` foi parametrizada, assegurando conexões estritamente por meio do protocolo HTTPS e abortando degradação (*downgrade*) para conexões abertas, preservando a imutabilidade dos *cookies* e mitigando invasões *Man-in-the-Middle*.
4. **Substituição de Tratamento Genérico de Erro:** Todo uso irresponsável de cláusulas amplas como `except Exception`, comuns a ferramentas automáticas, foi expurgado, favorecendo exceções modulares específicas (ex. falhas de DB, chamadas `pyOTP`, etc). Adicionalmente, hiperligações rígidas no sistema foram abstraídas por meio da engenharia reversa das rotas nativas com o emparelhamento dinâmico de `build_absolute_uri()` e `reverse()`, saneando, assim, os vazamentos e as amarras geradas por IA no fluxo da infraestrutura web.

6 Conclusão

As provas técnicas documentadas atestam que o protótipo funcional GSencript transcende o estigma da centralização de credenciais, utilizando os preceitos rigorosos do padrão *Privacy by Design*. A integração de algoritmos resistentes a ataques em hardware (*Argon2id*), simetria criptográfica dinâmica em repouso com *Zero-Knowledge*, e o controle irrestrito da persistência perante a Lei Geral de Proteção de Dados solidificam a arquitetura da plataforma. Conclui-se, após o saneamento de vulnerabilidades através da implementação de limitadores HTTP, servidores WSGI consolidados e blindagem lógica contra interceptações, que o GSencript configura-se como um *software* resiliente e maduro,

pronto para validação, homologação e eventual emprego integral em ecossistemas de produção corporativos.

Referências

- [1] ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. *ABNT NBR ISO/IEC 27001: Segurança da informação, segurança cibernética e proteção de privacidade - Sistemas de gestão da segurança da informação - Requisitos*. Rio de Janeiro: ABNT, 2022.
- [2] BRASIL. Lei nº 13.709, de 14 de agosto de 2018. *Lei Geral de Proteção de Dados Pessoais (LGPD)*. Diário Oficial da União, Brasília, DF, 15 ago. 2018.
- [3] OWASP. *OWASP Top 10: 2021*. The Open Worldwide Application Security Project, 2021. Disponível em: <<https://owasp.org/Top10/>>. Acesso em: 03 jun. 2026.
- [4] MARTIN, Robert C. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. 1. ed. Boston: Prentice Hall, 2017.